

university of groningen

Performance Engineering: Lecture #09

WMCS045-05

Dirk Pleiter

Bernoulli Institute

2026-09-28

Overview



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

Content



1. **Auto-Vectorization (2)**
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

Case Study: Matrix-Vector Multiplication (`matmulv`)



- **Purpose** of this case study: Vectorization of loops with loop carry dependencies
- **Specification**: Implement multiplication of a matrix and a vector:

$$\vec{y} \leftarrow A\,\vec{x}$$

  - A is a square matrix of size $N = 2048$
  - Matrix and vector elements are double-precision numbers

matmulv: Scalar Implementation



Benchmark routine:

```
1  void matmulv(double *restrict y,
2               double *restrict a,
3               double *restrict x,
4               const int n) {
5    for (int i = 0; i < n; i++) {
6      y[i] = 0.0;
7      for (int j = 0; j < n; j++)
8        y[i] += a[i * n + j] * x[j];
9    }
10 }
```

Benchmark driver:

```
1  #include <stdlib.h>
2
3  void matmulv(double *restrict,
4               double *restrict,
5               double *restrict,
6               const int);
7
8  int main() {
9    int n    = 2048;
10   double *a = (double *) calloc(n*n, sizeof(double));
11   double *x = (double *) calloc(n,   sizeof(double));
12   double *y = (double *) calloc(n,   sizeof(double));
13   for (int k = 0; k < 2000; k++)
14     matmulv(y, a, x, n);
15 }
```

`matmulv`: Vectorized



- Manual unrolling by $4\times$
- Compilation using
  `gcc -O3 -march=native`

Benchmark routine:

```
1  void matmulv(double *restrict y,
2               double *restrict a,
3               double *restrict x,
4               const int n) {
5    for (int i = 0; i < n; i++) {
6      double s[4] = {0.0, 0.0, 0.0, 0.0};
7      for (int j = 0; j < n; j += 4) {
8        s[0] += a[i * n + j+0] * x[j+0];
9        s[1] += a[i * n + j+1] * x[j+1];
10       s[2] += a[i * n + j+2] * x[j+2];
11       s[3] += a[i * n + j+3] * x[j+3];
12     }
13     y[i] = s[0] + s[1] + s[2] + s[3];
14   }
15 }
```

Assembler:

```
21 .L3:
22          xorl    %eax, %eax
23          xorl    %edx, %edx
24          vxorpd  %xmm0, %xmm0, %xmm0
25          .p2align 5
26          .p2align 4,,10
27          .p2align 3
28 .L4:
29          vmovupd (%r8,%rax), %ymm3
30          incl    %edx
31          vfmadd231pd     (%rsi,%rax), %ymm3, %ymm0
32          addq    $32, %rax
33          cmpl    %ecx, %edx
34          jb      .L4
35          vextractf64x2   $1, %ymm0, %xmm2
36          vunpckhpd       %xmm0, %xmm0, %xmm1
37          addq    $8, %rdi
38          addq    %r9, %rsi
39          vaddsd  %xmm0, %xmm1, %xmm1
40          valignq $3, %ymm0, %ymm0, %ymm0
41          vaddsd  %xmm2, %xmm1, %xmm1
42          vaddsd  %xmm0, %xmm1, %xmm0
43          vmovsd  %xmm0, -8(%rdi)
44          cmpq    %rdi, %r10
45          jne     .L3
```

`matmulv`: Vectorized with Outer-Loop Unrolling



Benchmark routine:

```
1  void matmulv(double *restrict y,
2               double *restrict a,
3               double *restrict x,
4               const int n) {
5    for (int i = 0; i < n; i += 4) {
6      y[i + 0] = y[i + 1] = y[i + 2] = y[i + 3] = 0.0;
7      for (int j = 0; j < n; j++) {
8        y[i+0] += a[(i+0) * n + j] * x[j];
9        y[i+1] += a[(i+1) * n + j] * x[j];
10       y[i+2] += a[(i+2) * n + j] * x[j];
11       y[i+3] += a[(i+3) * n + j] * x[j];
12     }
13   }
14 }
```

- **Issue**: Requires gather load for A

`matmulv`: Results on Intel i7-1165G7

	N_{cycle}
`-O1`	$33.3 \cdot 10^9$
`-O3`	$33.2 \cdot 10^9$
`-O3 -march=native`	$32.9 \cdot 10^9$
unroll4, `-O1`	$9.0 \cdot 10^9$
unroll4, `-O3`	$8.4 \cdot 10^9$
unroll4, `-O3 -march=native`	$8.1 \cdot 10^9$
unroll8, `-O3 -march=native`	$4.2 \cdot 10^9$
unroll4-outer, `-O3 -march=native`	$10.7 \cdot 10^9$

- **Compiler**: GCC 14.2.0
- **Observation**: Latency reduces by up to $7.9\times$
- **Cross-check**: Best performance at 4 Flop/cycle ✔

Mathematical Functions



- **Challenge**: Standards for languages like C foresee a scalar API for the mathematical functions and, therefore, make it difficult to exploit SIMD/vector instructions
  - Example:

$$\vec{y} \leftarrow \sin(\vec{x})$$

- **Possible solutions**:
  - Use programming models supporting mathematical functions, e.g. nsimd, Highway
  - Rely on built-in implementations provided by the compiler

Case Study: `sinf`



- **Purpose** of this case study: Check the impact of vectorizing mathematical functions
- **Specification**: Compute the sin function of an array of single-precision floating-point numbers

sinf: Code



Benchmark routine:

```c
#include <math.h>

void vsinf(float *restrict y,
           float *restrict x,
           int n)
{
  for (int i = 0; i < n; i++)
    y[i] = sinf(x[i]);
}
```

Benchmark driver:

```c
#include <stdlib.h>

void vsinf(float *restrict,
           float *restrict,
           int);

int main() {
  int n    = 16384;
  float *x = (float *) calloc(n, sizeof(float));
  float *y = (float *) calloc(n, sizeof(float));

  for (int k = 0; k < 100000; k++)
    vsinf(y, x, n);
}
```

`sinf`: Auto-Vectorized



Compilation using `gcc -O3 -ffast-math`

Benchmark routine:

```
1 #include <math.h>
2
3 void vsinf(float *restrict y,
4            float *restrict x,
5            int n)
6 {
7   for (int i = 0; i < n; i++)
8     y[i] = sinf(x[i]);
9 }
```

Assembler:

```
46 .L4:
47         movups  (%rbx), %xmm0
48         addq    $16, %rbx
49         addq    $16, %rbp
50         call    _ZGVbN4v_sinf@PLT
51         movups  %xmm0, -16(%rbp)
52         cmpq    %r12, %rbx
53         jne     .L4
```

`sinf`: Results on Intel i7-1165G7



	N_{cycle}	N_{instr}
`-O1`	$1.5 \cdot 10^{10}$	$3.9 \cdot 10^{10}$
`-O3`	$1.7 \cdot 10^{10}$	$3.9 \cdot 10^{10}$
`-O3 -ffast-math`	$1.0 \cdot 10^{10}$	$2.5 \cdot 10^{10}$

- **Compiler**: GCC 14.2.0
- **Observations**:
  - Latency and number of instructions reduces by about $1.5\times$ comparing `-O1` and `-O3 -ffast-math`
- **Crosschecks**:
  - Benchmark is repeated $10^5\times$ using $N = 16384$ ☞ 36 cycles per call of `sinf` ✔

Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

SIMD Programming Models: Introduction



- **Alternative** to auto-vectorization: **Manual vectorization**
- **Options**:
  - In-line assembler (not recommended)
  - Intrinsics
  - SIMD programming models: NSIMD, Highway, ...

Compiler Intrinsics



- **Compiler intrinsics** = Special functions that look like regular functions but are directly implemented by the compiler
- **Target benefit(s)**:
  - Improved performance due to stronger control on code generation
- **Possible drawback(s)**:
  - Compiler- and/or ISA-specific ☞ non-portable code

Intrinsics for AVX-512



- Documentation: Intel Intrinsics Guide
- Selected new data types: `__m512d`
- Selected intrinsics:

`_mm512_loadu_pd`:	Load unaligned doubles
`_mm512_set1_pd`:	Broadcast doubles
`_mm512_mul_pd`:	Multiply packed doubles
`_mm512_storeu_pd`:	Store unaligned doubles

- GCC compilation may require the following options: `-mavx512f -mavx512dq -mfma`

Vector Scale: AVX-512 Intrinsics



Compilation using `gcc-14 -O3 -mavx512f`

Benchmark routine:

```
1  #include <immintrin.h>
2
3  void vscale_avx512(double *restrict y,
4                     const double *restrict x,
5                     double a, int n)
6  {
7    __m512d va = _mm512_set1_pd(a);
8
9    for (int i = 0; i < n; i += 8) {
10       __m512d vx = _mm512_loadu_pd(&x[i]);
11       __m512d vy = _mm512_mul_pd(va, vx);
12       _mm512_storeu_pd(&y[i], vy);
13   }
14 }
```

Assembler:

```
6  vscale_avx512:
7  .LFB6571:
8          .cfi_startproc
9          endbr64
10         vbroadcastsd    %xmm0, %zmm0
11         testl   %edx, %edx
12         jle     .L5
13         xorl    %eax, %eax
14         .p2align 5
15         .p2align 4
16         .p2align 3
17 .L3:
18         vmulpd  (%rsi,%rax,8), %zmm0, %zmm1
19         vmovupd %zmm1, (%rdi,%rax,8)
20         addq    $8, %rax
21         cmpl    %eax, %edx
22         jg      .L3
23 .L5:
24         vzeroupper
25         ret
26         .cfi_endproc
```

Intrinsics for NEON



- Documentation: Arm Neon Intrinsics Reference
- Selected new data types: `float64x2_t`
- Selected intrinsics:

  `vdupq_n_f64`: Duplicate vector element to vector
  `vld1q_f64`: Load multiple single-element structures
  `vmulq_f64`: Multiply packed doubles
  `vst1q_f64`: Store multiple single-element structures

Vector Scale: NEON Intrinsics



Compilation using gcc-14 -O3

Benchmark routine:

```
1  #include <arm_neon.h>
2
3  void vscale_neon(double *restrict y,
4                   const double *restrict x,
5                   double a, int n)
6  {
7    float64x2_t va = vdupq_n_f64(a);
8
9    for (int i = 0; i < n; i += 2) {
10       float64x2_t vx = vld1q_f64(&x[i]);
11       float64x2_t vy = vmulq_f64(va, vx);
12       vst1q_f64(&y[i], vy);
13   }
14 }
```

Assembler:

```
6           .global vscale_neon
7           .type   vscale_neon, %function
8  vscale_neon:
9  .LFB3916:
10          .cfi_startproc
11          dup     v0.2d, v0.d[0]
12          cmp     w2, 0
13          ble     .L1
14          sub     w3, w2, #1
15          mov     x2, 0
16          lsr     w3, w3, 1
17          add     w3, w3, 1
18          lsl     x3, x3, 4
19          .p2align 5,,15
20 .L3:
21          ldr     q31, [x1, x2]
22          fmul    v31.2d, v31.2d, v0.2d
23          str     q31, [x0, x2]
24          add     x2, x2, 16
25          cmp     x3, x2
26          bne     .L3
```

NSIMD



- NSIMD is a vectorization library for C and C++
- Documentation: github.io
- Large range of supported architectures ☞ portability
  - Intel AVX-512 for KNLs
  - Intel AVX-512 for Skylake processors
  - Arm NEON 128 bits (ARMv8 and later)
  - Arm SVE (original sizeless SVE)
  - ...
- Various vectorised fuctions
  - Trigonometric functions
  - Exponential and logarithmic functions
  - Shuffle functions
  - ...

Vector Scale: NSIMD



```cpp
#include <nsimd/nsimd-all.hpp>

using namespace nsimd;

void vscale_nsimd(double* y,
                  const double* x,
                  double a, int n)
{
  using vdouble = pack<double>;
  const auto size = len<vdouble>();
  vdouble va = set1<vdouble>(a);

  for (int i = 0; i < n; i += size) {
    vdouble vx = loadu<vdouble>(&x[i]);
    vdouble vy = mul(va, vx);
    storeu(&y[i], vy);
  }
}
```

Highway



- C++ library that provides portable SIMD/vector intrinsics
- Documentation: github.io
- Large range of supported architectures ☞ portability
  - `AVX10_2`
  - `NEON`
  - `RVV`
  - `...`

Vector Scale: Highway



```cpp
#include <hwy/highway.h>

namespace hw = hwy::HWY_NAMESPACE;

void vscale_highway(double* HWY_RESTRICT y,
                    const double* HWY_RESTRICT x,
                    double a, int n)
{
  const hw::ScalableTag<double> vdouble;
  const auto size = hw::Lanes(vdouble);
  auto va = hw::LoadU(vdouble, &a);

  for (int i = 0; i < n; i += size) {
    auto vx = hw::LoadU(vdouble, &x[i]);
    auto vy = hw::Mul(va, vx);
    hw::StoreU(vy, vdouble, &y[i]);
  }
}
```

Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. Final

Introduction to Branch Optimization



- Approaches to branch optimization:
  - Avoid branching
  - Help branch predictor
  - Software support

Case Study: Table Lookup



- **Purpose** of this case study: Explore branching in the context of table lookups
- **Specification** for `table_lookup`:
  - Implement a benchmark for random table lookups based on an index
  - For simplicity consider the index k being an integer number, i.e. $k = 0, 1, \ldots$

Table Lookup: Implementation with Branches


```c
#include <math.h>
#include <stdlib.h>

double x;

int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 2;
    switch (k) {
      case 0: x = 2.0; break;
      case 1: x = 2.1; break;
    }
  }
}
```

```c
#include <math.h>
#include <stdlib.h>

double x;

int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 10;
    switch (k) {
      case 0: x = 2.0; break;
      case 1: x = 2.1; break;
      case 2: x = 2.2; break;
      case 3: x = 2.3; break;
      case 4: x = 2.4; break;
      case 5: x = 2.5; break;
      case 6: x = 2.6; break;
      case 7: x = 2.7; break;
      case 8: x = 2.8; break;
      case 9: x = 2.9; break;
    }
  }
}
```

Branch Optimization: Table Lookup



- **Branch optimization using table lookups** = Technique that replaces conditional branches (like long if-else or switch statements) with a lookup into an array or dictionary
- **Target benefit(s)**
  - Eliminate branches
- **Possible drawback(s)**
  - Requires storing the lookup table and performing table lookups
  - More data cache misses

Table Lookup: Implementation without Branches



```c
#include <math.h>
#include <stdlib.h>

double x;
const double table[] = { 2.0, 2.1 };

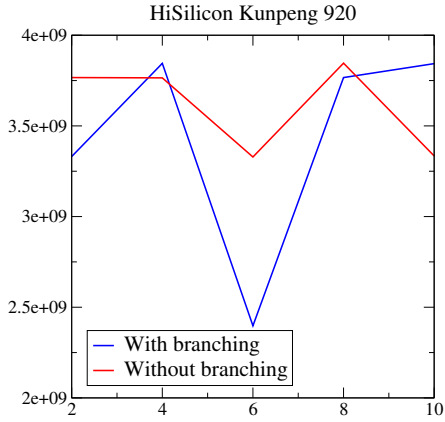
int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 2;
    x = table[k];
  }
}
```

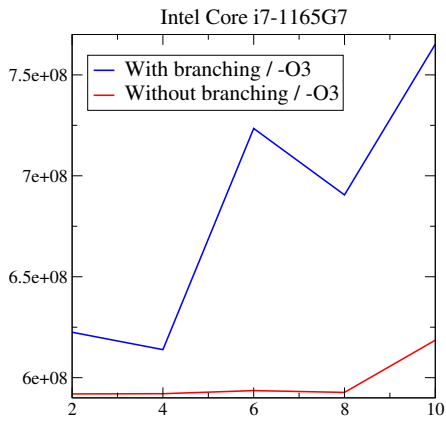
```c
#include <math.h>
#include <stdlib.h>

double x;
const double table[] =
    { 2.0, 2.1, 2.2, 2.3, 2.4,
      2.5, 2.6, 2.7, 2.8, 2.9 };

int main() {
  for (int i = 0; i < 10000000; i++) {
    int k = rand() % 10;
    x = table[k];
  }
}
```

Table Lookup: Results





Other Branch Optimizations



- **Help the branch predictor**, e.g., by
  - Keeping branch patterns stable
  - Reduce branch frequency and branching locations to improve use of Branch History Tables
- **Software support for branch prediction**
  - Branch prediction hints
    - Support has very much reduced
  - Arm v8 (aarch32) supports branch predictor maintenance
  - GCC supports identification of the most likely taken branch (`__builtin_expect()`)

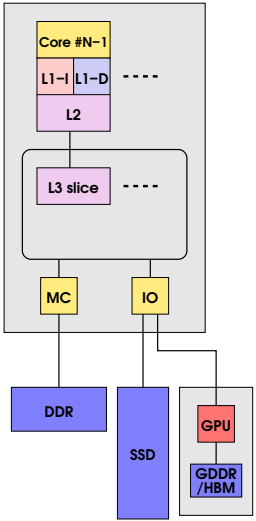
Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. **Memory Architecture (1)**
5. Final

Memory in Modern Node Architectures

- **Today's compute nodes: different memory types and technologies**
  - **Volatile memories**
    - Main memory (DDR3, DDR4, or DDR5)
    - Caches (SRAM)
    - Accelerator memory (GDDR or HBM)
  - **Non-volatile memories**
    - SSD (NAND flash)
- **Different capabilities**
  - Bandwidth
  - Latency for single transfer
    - Access time
    - Cycle time
  - Capacity



Memory Technologies: DRAM (1/2)



- Data bit storage cell $=$ 1 capacitor $+$ 1 transistor
- Capacitors discharge ☞ periodic **refresh** needed
  - Refresh required in intervals of O(10) ms
  - Possibly significant performance impact
  - Memory access latencies become variable
- DRAM typically packaged on small boards: **DIMM** $=$ Dual Inline Memory Modules
  - ≥ 1 DIMM per memory channel
  - Typical DRAM channel width: $N_{\mathrm{width}} = 64$ bit

Memory Technologies: DRAM (2/2)



- How to determine a theoretic upper bound for memory bandwidth:

$$B_{\text{mem}} = N_{\text{socket}} \cdot N_{\text{channel}} \cdot N_{\text{width}} \cdot B_{\text{rate}}$$

N_{socket}:	Number of sockets
N_{channel}:	Number of memory channels per socket
N_{width}:	Memory channel width
B_{rate}:	Data rate

- Performance range for selected memory technologies:

	DDR3	DDR4	DDR5
Standards release	2007	2012	2020
Data rate [Gbps]	$0.4 - 2.333$	$2.133 - 3.2$	$4 - 8.8$
Bandwidth/DIMM [GByte/s]	$3.2 - 18.6$	$17.1 - 25.6$	$32 - 70.4$

Memory Technologies: SRAM



- Data bit storage cell = (4 + 2) transistors
- Comparison with DRAM

	DRAM	SRAM
Refresh required?	yes	no
Speed	slower	faster
Density	higher	lower

- Typically used for on-chip memory

Data Locality (1/3)



- **Temporal locality** = There is a high probability for a memory location to be accessed again in the near future
  - Observe memory accesses during a time window W_{t}:

$$A_t, A_{t+1}, A_{t+2}, \ldots, A_{t+W_{\mathrm{t}}-1}$$

  - Let $\rho_{\mathrm{t}}(B)$ be the fraction of addresses with

$$A_{t+i} \equiv B$$

    for $B \in \{A_t, \ldots, A_{t+W_{\mathrm{t}}-1}\}$ and $0 \leq i < W_{\mathrm{t}}$
  - **Temporal locality** means that for some B we find $\rho_{\mathrm{t}}(B) \gg 1/W_{\mathrm{t}}$

Data Locality (2/3)



- **Spatial locality** = There is a high probability for a nearby memory location to be accessed in the near future
  - Observe memory accesses during a time window W_{t}:

$$A_t, A_{t+1}, A_{t+2}, \ldots, A_{t+W_{\mathrm{t}}-1}$$

  - Let $\rho_{\mathrm{s}}(\delta)$ be the fraction of pairs (A_{t+i}, A_{t+j}) $(0 \leq i < j < W_{\mathrm{t}})$ with

$$0 < |A_{t+i} - A_{t+j}| \leq \delta$$

    and $\delta > 0$
  - **Spatial locality** means that $\rho_{\mathrm{s}}(\delta)$ is large for small δ

Data Locality (3/3)



Question: Which type of locality applies to which pattern?

Time	Pattern #1	Pattern #2
0	$A + 0$	$A + 0$
1	$A + 1$	$A + 4096$
2	$A + 2$	$A + 0$
3	$A + 3$	$A + 4096$
4	$A + 4$	$A + 8192$
5	$A + 5$	$A + 0$
6	$A + 6$	$A + 4096$
7	$A + 7$	$A + 0$
8	$A + 8$	$A + 0$
9	$A + 9$	$A + 16384$

Data Locality (3/3)



Question: Which type of locality applies to which pattern?

- Pattern #1: Spatial locality
  $\rho_{\mathrm{t}}(A+0) = 1/10$
  $\rho_{\mathrm{s}}(10) = 1$
- Pattern #2: Temporal locality
  $\rho_{\mathrm{t}}(A+0) = 5/10$
  $\rho_{\mathrm{t}}(A+4096) = 3/10$
  $\rho_{\mathrm{s}}(10) = 0$

Time	Pattern #1	Pattern #2
0	$A+0$	$A+0$
1	$A+1$	$A+4096$
2	$A+2$	$A+0$
3	$A+3$	$A+4096$
4	$A+4$	$A+8192$
5	$A+5$	$A+0$
6	$A+6$	$A+4096$
7	$A+7$	$A+0$
8	$A+8$	$A+0$
9	$A+9$	$A+16384$

Reuse Distance



- **Reuse distance** = The number of accesses to distinct addresses made since the last reference to the same address
- **Example**:

Memory address	d	a	c	b	c	c	d	e	a	c
Reuse distance	-	-	-	-	1	0	3	-	4	3

- **Application** of the reuse distance:
  - Assume a cache of size C with Least Recently Used (LRU) replacement policy
  - If reuse distance $d \leq C$ then the address will be a cache hit

Memory and Storage Hierarchy



Computers provide multiple levels of volatile memory and non-volatile storage

- Important because of the "**processor–memory performance gap**"
- Close to the processor: very fast memory, but less capacity

Level	Access time	Capacity
Registers	$O(0.1)$ ns	$O(1)$ Ki B
L1 cache	$O(1)$ ns	$O(10-100)$ Ki B
L2 cache	$O(10)$ ns	$O(100)$ Ki B
L3 cache	$O(10)$ ns	$O(1-100)$ Mi B
Memory	$O(100)$ ns	$O(10-100)$ Gi B
Disk	$O(10)$ ms	$O(100-\ldots)$ Gi B

Content



1. Auto-Vectorization (2)
2. SIMD Programming Models
3. Branch Optimizations
4. Memory Architecture (1)
5. **Final**

Summary Lecture #09



- **Auto-vectorization**:
  - Case study: matrix-vector multiplication
  - Vectorization of mathematical functions
- **Programming models**: Intrinsics, NSIMD, Highway
- **Branch optimizations**: Table lookup and others
- Performance relevant features of memory architectures
  - Memory technologies and their performance
  - Data locality
  - Memory and storage hierarchy

Questions?



[Wiki Media]

These slides contain content and figures from

- D. Bakhvalov, Performance Analysis and Tuning on Modern CPUs (2nd Ed.). 2024. [Online]. Available: `https://github.com/dendibakh/perf-book/releases`